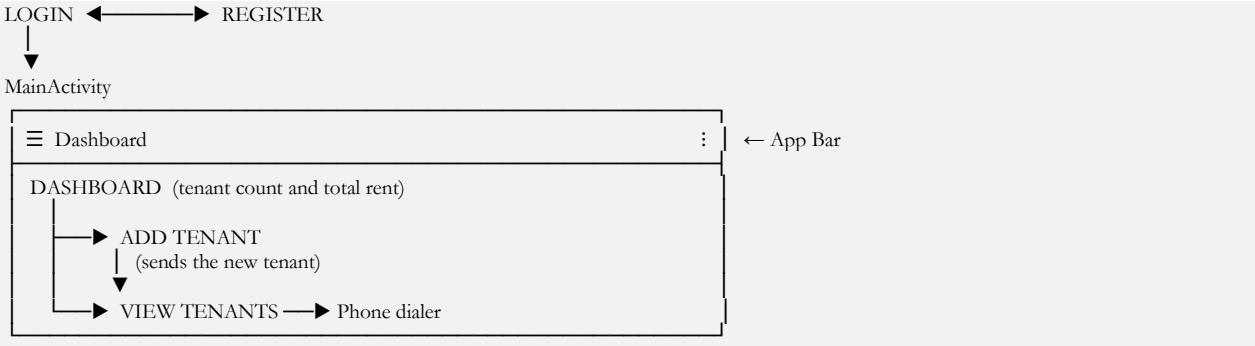
Strathmore
UNIVERSITY

SCHOOL OF COMPUTING AND ENGINEERING SCIENCES
BACHELOR IN BUSINESS INFORMATION TECHNOLOGY
BBT 3.2: Mobile Application Development
Language: Kotlin
Lesson 6
Navigation Practical 3 — Navigation, Safe Args and the Navigation Drawer
App Navigation, Part 3 of 3

What we are building

In Practical 2, you watched the Activity lifecycle, saved state across rotation, built the Dashboard, Add Tenant and View Tenants Fragments, and mapped them in nav_graph.xml.

In this practical, you connect everything: MainActivity shows the Fragments with an App Bar, the buttons move between screens, a new tenant travels from Add Tenant to View Tenants, and users get an **options menu** (⋮) and a **navigation drawer** (☰):



We will work through:

1. Add a NavHost and an App Bar to MainActivity
2. Navigate with the NavController and pass data with Safe Args
3. Watch the Fragment lifecycle in Logcat
4. Customise navigation behaviour (back stack and animations)
5. Bring back the CALL TENANT button
6. Add an options menu with NavigationUI
7. Build a navigation drawer

This practical continues from **Practical 2**. You need the three Fragments, TenantRepository, a Serializable Tenant, and nav_graph.xml.

Throughout this guide, boxes marked **Question** answer the things students most often get stuck on. If something goes wrong, check the nearest Question box first, then the **Common errors** table at the end.

Everywhere this guide shows com.example.tenantmanagementsystemgroupa, use your own package name.

> **Note: Read the code, don't just copy and paste.**
> Every code block has comments explaining what the code does and how the pieces connect. Read them before you paste, and make sure you can explain each line.

PART A — Add the NavHost and App Bar to MainActivity

MainActivity currently shows the old Add Tenant form. We turn it into the frame that holds our Fragments, with an App Bar at the top.

Step 1 — Replace activity_main.xml

Open activity_main.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

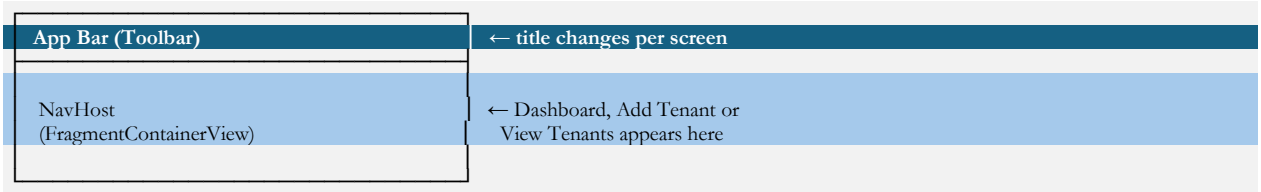
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:fitsSystemWindows="true">

   <com.google.android.material.appbar.MaterialToolbar
      android:id="@+id/toolbar"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:minHeight="?attr/actionBarSize"
      android:background="#E8EEF7" />

   <androidx.fragment.app.FragmentContainerView
      android:id="@+id/nav_host_fragment"
      android:name="androidx.navigation.fragment.NavHostFragment"
      android:layout_width="match_parent"
      android:layout_height="0dp"
      android:layout_weight="1"
      app:defaultNavHost="true"
      app:navGraph="@navigation/nav_graph" />

</LinearLayout>
```

The screen is now two parts:



Attribute	What it does
FragmentContainerView	The placeholder where Fragments are shown
android:name	Use Android's built-in NavHostFragment in this placeholder
app:navGraph	Which navigation graph to follow (our nav_graph.xml)
app:defaultNavHost="true"	Let this host handle the phone's Back button

> **Question: What happens to the form and the CALL TENANT button from Practical 1?**
> The form now lives in fragment_add_tenant.xml (Practical 2, Step 13). The CALL TENANT button moves to the View Tenants screen in Part E.
>
> **Question: What happens to lastTenant and onSaveInstanceState() from Practical 2, Part B?**
> They go too, because MainActivity no longer holds a tenant. The Fragments keep their own data across rotation, and Part C shows you how. The lifecycle log methods stay: they're included in the new MainActivity below.

> **Question: The old activity_main.xml had <layout> and a tenant variable. Where did they go?**
> MainActivity no longer shows tenant data, so it doesn't need Data Binding. It's a normal layout again, and it still gets an ActivityMainBinding through View Binding. The Data Binding now lives in fragment_view_tenants.xml.

Step 2 — Replace MainActivity.kt

Open MainActivity.kt. Delete everything **except the package line**, then paste:

```
import android.os.Bundle
import android.util.Log
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.NavController
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.AppBarConfiguration
import androidx.navigation.ui.navigateUp
import androidx.navigation.ui.setupActionBarWithNavController
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    // The "driver" that switches Fragments inside the NavHost
    private lateinit var navController: NavController

    // Tells the App Bar which screens are top-level (no back arrow)
    private lateinit var appBarConfiguration: AppBarConfiguration

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("TENANT_LIFECYCLE", "MainActivity: onCreate")

        // Create the views from activity_main.xml (Toolbar + NavHost)
        binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        // 1. Use our Toolbar as the App Bar
        //    toolbar ← android:id="@+id/toolbar" in activity_main.xml
        setSupportActionBar(binding.toolbar)

        // 2. Get the NavController from the NavHost
        //    nav_host_fragment ← android:id="@+id/nav_host_fragment"
        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment

        navController = navHostFragment.navController

        // 3. Connect the App Bar to navigation:
        //    the title comes from each destination's android:label
        appBarConfiguration = AppBarConfiguration(navController.graph)

        setupActionBarWithNavController(navController, appBarConfiguration)
    }

    // 4. Make the App Bar's back arrow work
    override fun onSupportNavigateUp(): Boolean {
        return navController.navigateUp(appBarConfiguration) || super.onSupportNavigateUp()
    }

    // ── LIFECYCLE LOGS ──────────────────────────────────────────────────
    // Keep your onStart(), onResume(), onPause(), onStop(),
    // onRestart() and onDestroy() methods from Practical 2, Part A here.
    // (They are not repeated in this guide to save space.)
}
```

What each numbered part does:

1. **setSupportActionBar** tells Android to use our Toolbar as the App Bar, so it can show titles and menus.
2. The **NavController** is the driver: it knows the map (nav graph) and switches screens inside the NavHost.

3. **setupActionBarWithNavController** is NavigationUI. It updates the App Bar title from each destination's android:label, and shows a back arrow (←) on every screen except the start destination.
4. **onSupportNavigateUp** makes that back arrow actually go back.

> **Question: The app crashes with "This Activity already has an action bar supplied by the window decor".**
> Your theme already provides an App Bar, so Android refuses to add ours. Open res → values → themes.xml and make sure the theme's parent ends in .NoActionBar, for example Theme.Material3.DayNight.NoActionBar. Check values-night/themes.xml too if it exists.
>
> **Question: MaterialToolbar is red in the XML.**
> Your project is missing the Material library. In the Module Gradle file's dependencies, add implementation("com.google.android.material:material:1.12.0") and sync.
>
> **Question: What is the difference between the back arrow in the App Bar and the phone's Back button?**
> In our app they do the same thing. The App Bar arrow is called **Up**; the phone button is called **Back**. app:defaultNavHost="true" connects the phone's Back button, and onSupportNavigateUp connects the Up arrow.

Checkpoint. Run the app and log in. You should see the **Dashboard** with "Dashboard" in the App Bar, and metrics showing 0 tenants. The buttons don't do anything yet.

PART B — Navigate with the NavController and Safe Args

Step 3 — Build the project to generate the Safe Args classes

From the menu at the top, choose:

```
Build → Assemble Project
```

(or press **Ctrl+F9**, Mac: **Cmd+F9**).

Safe Args reads your navigation graph and generates new classes for you:

```
nav_graph.xml
    │
    ▼ (Safe Args)
DashboardFragmentDirections     ← one function per action from Dashboard
AddTenantFragmentDirections     ← one function per action from Add Tenant
ViewTenantsFragmentArgs         ← gives View Tenants its arguments
```

> **Question: Where are these files? I can't see them.**
> They are generated during the build and hidden from the normal Project view. You don't edit them; you just use them in your code. Android Studio will recognise the names once the build finishes.

Step 4 — Make the Dashboard buttons navigate

Open DashboardFragment.kt. Add this import at the top:

```
import androidx.navigation.fragment.findNavController
```

Inside onViewCreated(), underneath the two lines that set the metric text, add:

```
    binding.addTenantButton.setOnClickListener {

        val action = DashboardFragmentDirections.actionDashboardFragmentToAddTenantFragment()

        findNavController().navigate(action)
    }

    binding.viewTenantsButton.setOnClickListener {

        val action = DashboardFragmentDirections.actionDashboardFragmentToViewTenantsFragment()

        findNavController().navigate(action)
    }
```

Look at how the names are built from the res/navigation/ nav graph:

You will notice all fragments were listed there and their action (from where to where)

```
<fragment android:id="@+id/dashboardFragment">          →  DashboardFragmentDirections
<action android:id="@+id/action_dashboardFragment_to_addTenantFragment">
                                        →  .actionDashboardFragmentToAddTenantFragment()
```

Question: What is findNavController()?
It finds the NavController that owns this Fragment (the one in MainActivity's NavHost). Defining an action only names the path; calling navigate(action) on the NavController is what actually moves to the next screen.

Question: DashboardFragmentDirections is red.
Run **Build → Make Project** again (Step 3). If it is still red, check that the action IDs in nav_graph.xml are spelled exactly as shown, and that Safe Args is set up in both Gradle files.

Question: Why doesn't actionDashboardFragmentToViewTenantsFragment() need a tenant?
Because we gave the newTenant argument a default of @null. When you don't pass one, it is null.

Step 5 — Make SAVE add the tenant and pass it to View Tenants

Open AddTenantFragment.kt. Add these imports at the top:

```
import android.widget.Toast
import androidx.navigation.fragment.findNavController
```

Replace the comment // The SAVE button is added in Part B with:

```
    // saveButton ←  android:id="@+id/saveButton" in fragment_add_tenant.xml
    binding.saveButton.setOnClickListener {

        // Read the three EditTexts in fragment_add_tenant.xml
        val name = binding.tenantNameEditText.text.toString().trim()

        val phone = binding.phoneEditText.text.toString().trim()

        val rent = binding.rentEditText.text.toString().trim()

        if (name.isEmpty() || phone.isEmpty() || rent.isEmpty()) {
            // A Fragment is not a Context, so ask for its Activity's
            Toast.makeText(requireContext(), "Please fill in all fields", Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }

        val tenant = Tenant(name, phone, rent)

        // Add to the shared list (the Dashboard and View Tenants read it)
        TenantRepository.addTenant(tenant)

        // SAFE ARGS: this function was generated from the <action> in
        // nav_graph.xml. Because View Tenants has a <argument> of type
        // Tenant, the function REQUIRES a Tenant: it won't compile without one
        val action = AddTenantFragmentDirections
            .actionAddTenantFragmentToViewTenantsFragment(tenant)

        // Ask the NavController to follow the action
        findNavController().navigate(action)
    }
```

The tenant travels with the action:

```
AddTenantFragment                    ViewTenantsFragment
    |                                        ▲
    |  actionAddTenantFragmentToViewTenantsFragment(tenant)
    └────────────────────────────────────────┘
            newTenant = Tenant("John Kamau", ...)
```

Question: Why requireContext() and not this for the Toast?
A Toast needs a Context. An Activity is a Context, so in an Activity we use this. A Fragment is not a Context, so we ask for the one it's attached to with requireContext().

> **Question: Why is Safe Args better than putExtra("EMAIL", email) like we did with Activities?**
> With putExtra, a typo in the key ("EMAIL" vs "Email") or the wrong type is only discovered when the app runs, as missing data or a crash. With Safe Args, the function only accepts a Tenant, so a mistake is a red error before you run.

Step 6 — Receive the tenant in View Tenants

Open ViewTenantsFragment.kt. Add this import at the top:

```
import androidx.navigation.fragment.navArgs
```

Underneath private val binding get() = _binding!!, add:

```
    private val args: ViewTenantsFragmentArgs by navArgs()
```

Inside onViewCreated(), **replace** these two lines:

```
        // For now, hide the "Just added" section. Part B shows it when a tenant is passed in.
        binding.newTenantGroup.visibility = View.GONE
```

with:

```
        val newTenant = args.newTenant

        binding.tenant = newTenant

        if (newTenant == null) {
            binding.newTenantGroup.visibility = View.GONE
        }
```

Your complete ViewTenantsFragment.kt:

```
package com.example.tenantmanagementsystemgroupa

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.navArgs
// Generated from fragment_view_tenants.xml (a Data Binding layout)
import com.example.tenantmanagementsystemgroupa.databinding.FragmentViewTenantsBinding

class ViewTenantsFragment : Fragment() {

    private var _binding: FragmentViewTenantsBinding? = null
    private val binding get() = _binding!!

    // SAFE ARGS: reads the arguments that arrived with the action.
    // ViewTenantsFragmentArgs was generated from the <argument> in nav_graph.xml
    private val args: ViewTenantsFragmentArgs by navArgs()

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentViewTenantsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // The tenant sent from Add Tenant, or null if we came from the Dashboard
        // (newTenant ← <argument android:name="newTenant"> in nav_graph.xml)
        val newTenant = args.newTenant

        // DATA BINDING: give it to <variable name="tenant"> in the layout.
        // newTenantTextView shows @{tenant.summary()}, so it updates itself
        binding.tenant = newTenant

        // No new tenant? Hide "Just added", the box and the Call button together
        // newTenantGroup ← the <Group> in fragment_view_tenants.xml
        if (newTenant == null) {
```

```
            binding.newTenantGroup.visibility = View.GONE
        }

        // Show every saved tenant as text, one after another
        val tenants = TenantRepository.tenants

        if (tenants.isEmpty()) {
            binding.allTenantsTextView.text = "No tenants yet. Add one from the Dashboard."
        } else {
            binding.allTenantsTextView.text = tenants.joinToString("\n\n") { it.summary() }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

How the tenant travels with Safe Args

Three files work together. The **nav graph** declares the path and the data; Safe Args **generates** two classes from it; the two Fragments use those classes.

```
nav_graph.xml
─────────────────────
<fragment android:id="@+id/addTenantFragment">
    <action
        android:id="@+id/action_addTenantFragment_to_viewTenantsFragment"
        app:destination="@id/viewTenantsFragment" />        ── 1. the path
</fragment>

<fragment android:id="@+id/viewTenantsFragment">
    <argument
        android:name="newTenant"                          ── 2. the data the
        app:argType="...Tenant" />                           destination expects
</fragment>
        │
        │  Build → Make Project: Safe Args generates
        ▼
AddTenantFragmentDirections                ViewTenantsFragmentArgs
  .actionAddTenantFragment                   .newTenant
    ToViewTenantsFragment(newTenant)
      │                                    ▲
      ▼                                    │
AddTenantFragment (sending)                ViewTenantsFragment (receiving)
──────────────────────────────────────     ──────────────────────────────────────
val action = AddTenantFragmentDirections    private val args:
    .actionAddTenantFragment                    ViewTenantsFragmentArgs by navArgs()
    ToViewTenantsFragment(tenant)
                                            val newTenant = args.newTenant
findNavController().navigate(action)  ──▶   binding.tenant = newTenant
```

Compare this with `putExtra` in Practical 1:

	Intent extras (Practical 1)	Safe Args (today)
Sending	`intent.putExtra("EMAIL", email)`	`...ToViewTenantsFragment(tenant)`
Receiving	`intent.getStringExtra("EMAIL")`	`args.newTenant`
Key	A String you type on both sides	Declared once in `nav_graph.xml`
Typo in the key	Compiles, then silently gives `null`	Red error before you run
Sending the wrong type	Compiles, then wrong data or a crash	Red error before you run

> **Question: How can Safe Args send a whole Tenant object?**
> Because `Tenant` is **Serializable** (Practical 2, Step 9). When you navigate, Safe Args packs the Tenant into the destination Fragment's arguments, and `navArgs()` unpacks it on the other side. Without `: Serializable` on the class, the nav graph's `app:argType="...Tenant"` would not build.

> **Question: Does sending a Serializable object also explain why it survives rotation?**
> Yes. A Fragment's arguments are saved and restored automatically with the Fragment. Because the Tenant is packed into those arguments, it comes back after a rotation with no onSaveInstanceState() code. You'll see this in Part C.
>
> **Question: What does by navArgs() do?**
> It reads the arguments that were sent with the action and gives them to you as a ViewTenantsFragmentArgs object. args.newTenant is the tenant from Add Tenant, or null if we came from the Dashboard.
>
> **Question: binding.tenant = newTenant — where have I seen this before?**
> In the last practical. fragment_view_tenants.xml has a tenant variable, and @{tenant.summary()} displays it. Data Binding works the same way in a Fragment.

Checkpoint. Run the app and log in.

1. Tap **VIEW TENANTS**: "No tenants yet", with no "Just added" section. Tap the back arrow.
2. Tap **ADD TENANT**, fill in a tenant and tap **SAVE**: View Tenants opens showing "Just added" and the tenant in the list.
3. Now press **Back**. Notice where you land.

PART C — Watch the Fragment lifecycle

Fragments have a lifecycle too. It looks like the Activity lifecycle, with one important difference: a Fragment's **view** (its screen) can be destroyed and rebuilt while the Fragment **object** stays alive.

```
        onCreate()          the Fragment object is created
            ↓
        ┌── onCreateView()      its VIEW is built from XML (binding created)
        │       ↓
        │   onViewCreated()     the view is ready: set text, click listeners
 view   │       ↓
lifecycle   onStart()  →  onResume()     visible and interactive
        │       ↓
        │   onPause()  →  onStop()
        │       ↓
        └── onDestroyView()     its VIEW is destroyed (binding cleared)
            ↓
        onDestroy()         the Fragment object is destroyed
```

Activity	Fragment	Difference
onCreate() builds the screen	onCreate() + onCreateView()	A Fragment builds its view in a separate method
—	onViewCreated()	Where we set text and click listeners in a Fragment
onStart(), onResume(), onPause(), onStop()	The same	Same meaning
—	onDestroyView()	The view is gone, but the Fragment may come back
onDestroy()	onDestroy()	Same meaning

Step 7 — Add lifecycle logs to DashboardFragment

Open DashboardFragment.kt. Add this import at the top:

```
import android.util.Log
```

Then replace the whole class (keep the package line and imports) with:

```
class DashboardFragment : Fragment() {

    private var _binding: FragmentDashboardBinding? = null
```

```
private val binding get() = _binding!!

// 1. The Fragment OBJECT is created (no view yet)
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onCreate")
}

// 2. Build the VIEW from fragment_dashboard.xml
override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View {
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onCreateView")
    _binding = FragmentDashboardBinding.inflate(inflater, container, false)
    return binding.root
}

// 3. The view exists: fill in the metrics and set click listeners
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onViewCreated")

    val tenantCount = TenantRepository.tenants.size
    val totalRent = TenantRepository.totalRent()

    binding.tenantCountTextView.text = "Tenants\n$tenantCount"
    binding.totalRentTextView.text = "Total rent\nKSh %,.0f".format(totalRent)

    binding.addTenantButton.setOnClickListener {
        val action = DashboardFragmentDirections.actionDashboardFragmentToAddTenantFragment()
        findNavController().navigate(action)
    }

    binding.viewTenantsButton.setOnClickListener {
        val action = DashboardFragmentDirections.actionDashboardFragmentToViewTenantsFragment()
        findNavController().navigate(action)
    }
}

// 4. Visible
override fun onStart() {
    super.onStart()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onStart")
}

// 5. In front, user can interact
override fun onResume() {
    super.onResume()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onResume")
}

// 6. Leaving the front
override fun onPause() {
    super.onPause()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onPause")
}

// 7. No longer visible
override fun onStop() {
    super.onStop()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onStop")
}

// 8. The VIEW is destroyed. Clear the binding so the old view can be freed
override fun onDestroyView() {
    super.onDestroyView()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onDestroyView")
    _binding = null
}

// 9. The Fragment OBJECT is destroyed
override fun onDestroy() {
    super.onDestroy()
    Log.d("TENANT_LIFECYCLE", "   DashboardFragment: onDestroy")
```

```
    }
}
```

The Fragment's messages start with three spaces, so they stand out from MainActivity's in Logcat.

Step 8 — Watch both lifecycles together

Run the app, keep Logcat filtered on `tag:TENANT_LIFECYCLE`, and try each action.

1. Log in. The Activity and the Fragment start together. You'll see something like:

```
MainActivity: onCreate
  DashboardFragment: onCreate
  DashboardFragment: onCreateView
  DashboardFragment: onViewCreated
  DashboardFragment: onStart
MainActivity: onStart
MainActivity: onResume
  DashboardFragment: onResume
```

The exact order of the Activity and Fragment lines may differ slightly. What matters is that the Fragment can't be ahead of its Activity: it is only fully running once MainActivity is.

2. Tap ADD TENANT.

```
  DashboardFragment: onPause
  DashboardFragment: onStop
  DashboardFragment: onDestroyView
```

There is **no** `onDestroy`. The Dashboard's **view** is destroyed, but the Fragment object is kept on the back stack, because the user may come back to it.

3. Press Back.

```
  DashboardFragment: onCreateView
  DashboardFragment: onViewCreated
  DashboardFragment: onStart
  DashboardFragment: onResume
```

There is **no** `onCreate`. The same Fragment object builds a **new view**. Because `onViewCreated()` runs again, the metrics are recalculated, which is why the Dashboard shows new totals after you add a tenant.

4. Rotate the phone on the Dashboard. Everything is destroyed and recreated: MainActivity and DashboardFragment both show `onDestroy`, then `onCreate`.

5. Add a tenant, then rotate on View Tenants. The "Just added" tenant is still shown, and you didn't write any `onSaveInstanceState()` code for it.

> **Question: Why does View Tenants keep its tenant after rotating, with no extra code?**
> Safe Args stores the arguments in the Fragment's own saved state, and Android restores them automatically when the Fragment is recreated. This is one reason the Fragment version of our app no longer needs the `lastTenant` variable from Practical 2, Part B.
>
> **Question: So this is why we set `_binding = null` in `onDestroyView()`!**
> Exactly. Action 2 showed that the view is destroyed while the Fragment lives on. If `_binding` still pointed to the old view, that whole view would stay in memory for nothing.
>
> **Question: Can I add the logs to the other Fragments too?**
> Yes, and it's worth trying. Use the same pattern, with a different name in each message, for example "`AddTenantFragment: onPause`".

Checkpoint. You can explain why the Dashboard gets `onDestroyView` but not `onDestroy` when you navigate away from it.

PART D — Customise navigation behaviour

Step 9 — Fix the back stack with popUpTo

In the last checkpoint, pressing Back from View Tenants took you back to the **Add Tenant form**. That's not what a user expects after saving.

Android keeps a **back stack**: a pile of the screens you've visited, with the newest on top. Back removes the top one.

```
Without popUpTo                    With popUpTo dashboardFragment

  View Tenants   ← you are here      View Tenants   ← you are here
  Add Tenant     ← Back goes here    Dashboard      ← Back goes here
  Dashboard
```

Open nav_graph.xml (Code view). Find the action inside addTenantFragment:

```
<action
   android:id="@+id/action_addTenantFragment_to_viewTenantsFragment"
   app:destination="@id/viewTenantsFragment" />
```

Change it to:

```
<action
   android:id="@+id/action_addTenantFragment_to_viewTenantsFragment"
   app:destination="@id/viewTenantsFragment"
   app:popUpTo="@id/dashboardFragment" />
```

popUpTo="@id/dashboardFragment" means: "before going to View Tenants, remove every screen above the Dashboard from the back stack."

> **Question: What would app:popUpToInclusive="true" do?**
> It would remove the Dashboard as well. Pressing Back from View Tenants would then close the app. We don't want that here.

Step 10 — Add animations

In the same file, add slide animations to the two actions inside dashboardFragment:

```
<action
   android:id="@+id/action_dashboardFragment_to_addTenantFragment"
   app:destination="@id/addTenantFragment"
   app:enterAnim="@anim/nav_default_enter_anim"
   app:exitAnim="@anim/nav_default_exit_anim"
   app:popEnterAnim="@anim/nav_default_pop_enter_anim"
   app:popExitAnim="@anim/nav_default_pop_exit_anim" />

<action
   android:id="@+id/action_dashboardFragment_to_viewTenantsFragment"
   app:destination="@id/viewTenantsFragment"
   app:enterAnim="@anim/nav_default_enter_anim"
   app:exitAnim="@anim/nav_default_exit_anim"
   app:popEnterAnim="@anim/nav_default_pop_enter_anim"
   app:popExitAnim="@anim/nav_default_pop_exit_anim" />
```

Attribute	When it plays
enterAnim	The new screen appearing
exitAnim	The old screen leaving
popEnterAnim	The old screen coming back when you press Back
popExitAnim	The current screen leaving when you press Back

> **Question: Where do nav_default_enter_anim and the others come from? I don't have an anim folder.**
> They are built into the navigation library. You can also create your own in res/anim later.
>
> **Question: I can't see any animation.**
> Some emulators have animations turned off. On the emulator, go to **Settings → Developer options** and set the three "animation scale" options to **1x**.

Checkpoint. Run the app. Add a tenant, then press Back from View Tenants: you return to the **Dashboard**, and the metrics now show your tenant count and total rent. Moving from the Dashboard to another screen slides smoothly.

PART E — Bring back the CALL TENANT button

In Practical 1, the CALL TENANT button lived on the old Add Tenant screen. Now it lives on View Tenants, next to the tenant that was just added. The implicit Intent is the same as before.

Step 11 — Make CALL TENANT open the dialer

Open ViewTenantsFragment.kt. Add these imports at the top:

```
import android.content.Intent
import android.net.Uri
```

Inside onViewCreated(), underneath the if (newTenant == null) { ... } block. Add the code below **after the closing },** not inside it. The section should now look like this:

```
    if (newTenant == null) {

   binding.newTenantGroup.visibility = View.GONE
}
 binding.callButton.setOnClickListener {

        if (newTenant != null) {

           val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:${newTenant.phone}"))

           startActivity(intent)
        }
    }
```

Compare the two kinds of Intent in our app:

	Explicit Intent	Implicit Intent
In our app	Intent(this, MainActivity::class.java)	Intent(Intent.ACTION_DIAL, Uri.parse("tel:..."))
What it says	"Open this exact screen"	"I want to dial this number"
Who responds	Our own MainActivity	Whichever phone app is installed

> **Question: Does this make a phone call?**
> No. ACTION_DIAL opens the phone's dialer with the number already typed in. The user still has to press the call button. That's why it doesn't need any special permission.
>
> **Question: What is "tel:"?**
> It tells Android what kind of data this is: a telephone number. "tel:0712345678" is a phone number the same way "https://..." is a web address.

Checkpoint. Run the app, add a tenant, and tap **CALL TENANT** on View Tenants. The phone dialer opens with the tenant's number filled in.

NOTE: At this point your application Works perfectly but for a more professional look we will now add Menus and navigation drawer. This should take you approximately **10minutes**

Menus and the navigation drawer

The options menu and the navigation drawer both use a **menu resource**: an XML file in the res/menu folder that lists the items. Each item has an ID, a title and, optionally, an icon.

The key idea for today:

> **If a menu item's ID is the same as a destination ID in nav_graph.xml, NavigationUI can navigate there automatically, with no extra code.**

```
main_menu.xml / drawer_menu.xml          nav_graph.xml

<item android:id="@+id/viewTenantsFragment"   <fragment android:id="@+id/viewTenantsFragment"
    ...                                  ⟺       ...
```

PART F — The options menu

Step 12 — Create the menu resource

Right-click the **res** folder and choose **New → Android Resource File**:

```
File name:      main_menu
Resource type:  Menu
```

Click **OK**. Android Studio creates:

```
res
└── menu
    └── main_menu.xml
```

Open main_menu.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/viewTenantsFragment"
        android:title="View Tenants"
        app:showAsAction="never" />

    <item
        android:id="@+id/action_logout"
        android:title="Logout"
        app:showAsAction="never" />

</menu>
```

Look closely at the first item's ID: **viewTenantsFragment**. It is exactly the same as the destination ID in nav_graph.xml.

> **Question: What does showAsAction="never" mean?**
> The item always stays inside the ⋮ menu. With "ifRoom", Android would show it directly on the App Bar when there is space. With "always", it always appears on the App Bar.
>
> **Question: Why is Logout's ID action_logout and not a destination?**
> Logout isn't a screen inside MainActivity's navigation graph. It takes us back to LoginActivity, so we handle it ourselves with an Intent in Step 14.

Step 13 — Show the menu in the App Bar

Open MainActivity.kt. Add this import at the top:

```
import android.view.Menu
```

Underneath onSupportNavigateUp() (still inside the class), add:

```
// Turn main_menu.xml into menu items in the App Bar
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    menuInflater.inflate(R.menu.main_menu, menu)
    return true
}
```

> **Question: What does "inflate" mean for a menu?**
> The same as for a layout: turning the XML file into real menu items that appear on screen.
>
> **Question: Why return true?**
> It tells Android "yes, show this menu". Returning false would hide it.

Checkpoint. Run the app and log in. The ⋮ icon appears on the right of the App Bar. Tapping it shows **View Tenants** and **Logout**, but tapping them does nothing yet.

Step 14 — Handle menu taps

Still in MainActivity.kt, add these imports:

```
import android.content.Intent
import android.view.MenuItem
import androidx.navigation.ui.onNavDestinationSelected
```

Underneath onCreateOptionsMenu(), add:

```
// What happens when a menu item is tapped
override fun onOptionsItemSelected(item: MenuItem): Boolean {

    if (item.itemId == R.id.action_logout) {

        val intent = Intent(this, LoginActivity::class.java)

        intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK

        startActivity(intent)

        return true
    }

    return item.onNavDestinationSelected(navController) || super.onOptionsItemSelected(item)
}
```

How each menu item is handled:



> **Question: Why doesn't View Tenants need its own code?**
> That's what NavigationUI does. onNavDestinationSelected looks for a destination in the nav graph with the same ID as the menu item, and navigates there. Before NavigationUI, you'd write a separate if with a navigate(...) or an Intent for every item.
>
> **Question: Why does Logout use FLAG_ACTIVITY_NEW_TASK or FLAG_ACTIVITY_CLEAR_TASK?**
> Together they close every screen the app has open and start fresh with Login. Without them, pressing Back on the Login screen after logging out would take you back into the Dashboard.
>
> **Question: What does || mean in the last line?**
> "Or". If onNavDestinationSelected handled the item, it returns true and we're done. If not, we let Android's default handling try.

> **Question: I tap "View Tenants" and nothing happens.**
> The menu item's ID doesn't exactly match the destination ID in nav_graph.xml. Both must be viewTenantsFragment.

Checkpoint. Run the app.

1. From the Dashboard, open ⋮ and choose **View Tenants**. The View Tenants screen opens.
2. Open ⋮ and choose **Logout**. You're back on Login. Press Back: the app closes.

PART G — Send the user's email to MainActivity

The drawer header will show who is logged in. For that, MainActivity needs the email from the Login screen. You did this in Practical 1 with putExtra.

Step 15 — Attach the email in LoginActivity

Open LoginActivity.kt. Find the lines that open MainActivity:

```
        val intent = Intent(this, MainActivity::class.java)

        startActivity(intent)
```

Add one line between them:

```
        val intent = Intent(this, MainActivity::class.java)

        intent.putExtra("EMAIL", email)

        startActivity(intent)
```

We read it in MainActivity in Step 20.

PART H — Build the navigation drawer

A navigation drawer is made of four pieces:

```
DrawerLayout                ← the container that can slide a drawer in
├── your normal screen      ← App Bar + NavHost (what you have now)
└── NavigationView          ← the drawer itself
    ├── header (nav_header.xml)
    └── menu   (drawer_menu.xml)
```

Step 16 — Create the drawer menu

Right-click the **menu** folder (inside res) and choose **New → Menu Resource File**:

```
File name:  drawer_menu
```

Click **OK**. Open drawer_menu.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

  <group android:checkableBehavior="single">

    <item
      android:id="@+id/dashboardFragment"
      android:icon="@android:drawable/ic_menu_agenda"
      android:title="Dashboard" />

    <item
      android:id="@+id/addTenantFragment"
      android:icon="@android:drawable/ic_menu_add"
      android:title="Add Tenant" />

    <item
      android:id="@+id/viewTenantsFragment"
      android:icon="@android:drawable/ic_menu_view"
      android:title="View Tenants" />

  </group>

</menu>
```

All three IDs match destination IDs in nav_graph.xml: dashboardFragment, addTenantFragment and viewTenantsFragment.

> **Question: What does checkableBehavior="single" do?**
> Only one item in the group can be highlighted at a time. NavigationUI highlights the item for the screen you're on.
>
> **Question: Where do the icons come from?**
> @android:drawable/... icons are built into Android, so you don't need to add any image files. You can replace them with your own icons later using **New → Vector Asset**.
>
> **Question: viewTenantsFragment is used in both menus. Is that allowed?**
> Yes. IDs are shared across the whole app. Using the same ID in both menus means both lead to the same destination.

Step 17 — Create the drawer header

Right-click the **layout** folder and choose **New → Layout Resource File**:

```
File name:    nav_header
Root element:  LinearLayout
```

This designs how the app bar would look like.

Click **OK**. Open nav_header.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="160dp"
  android:orientation="vertical"
  android:gravity="bottom"
  android:padding="16dp"
  android:background="#1F4E79">

  <TextView
    android:id="@+id/headerTitleTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tenant Management System"
    android:textColor="#FFFFFF"
    android:textSize="18sp"
    android:textStyle="bold" />

  <TextView
```

```
        android:id="@+id/headerEmailTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="user@example.com"
        android:textColor="#FFFFFF"
        android:textSize="14sp"
        android:layout_marginTop="4dp" />

</LinearLayout>
```

> **Question: What does `gravity="bottom"` do?**
> It pushes the two TextViews to the bottom of the 160dp header, which is where drawer headers usually show the user's details.

Step 18 — Wrap activity_main.xml in a DrawerLayout

Open activity_main.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/drawerLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true">

    <!-- 1. The normal screen: App Bar + NavHost -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:fitsSystemWindows="true">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:minHeight="?attr/actionBarSize"
            android:background="#E8EEF7" />

        <androidx.fragment.app.FragmentContainerView
            android:id="@+id/nav_host_fragment"
            android:name="androidx.navigation.fragment.NavHostFragment"
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            app:defaultNavHost="true"
            app:navGraph="@navigation/nav_graph" />

    </LinearLayout>

    <!-- 2. The drawer -->
    <com.google.android.material.navigation.NavigationView
        android:id="@+id/navView"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        android:fitsSystemWindows="true"
        app:headerLayout="@layout/nav_header"
        app:menu="@menu/drawer_menu" />

</androidx.drawerlayout.widget.DrawerLayout>
```

The LinearLayout in the middle is exactly what you built in Part A. We've only wrapped it and added the NavigationView.

Part	What it does
DrawerLayout	Holds the screen and the drawer, and handles sliding the drawer in and out
First child (LinearLayout)	The normal screen, always visible

Part	What it does
NavigationView	The drawer panel
layout_gravity="start"	The drawer slides in from the left (the "start" side)
app:headerLayout	The layout shown at the top of the drawer
app:menu	The menu items listed in the drawer

> **Question: Does the order of the children matter?**
> Yes. The main screen must come **first** and the NavigationView **last**, so the drawer is drawn on top of the screen.
>
> **Question: What if I forget layout_gravity="start"?**
> DrawerLayout won't know it's a drawer. The NavigationView will cover your screen permanently.
>
> **Question: NavigationView is red.**
> It comes from the Material library. Check that com.google.android.material:material is in the Module Gradle file's dependencies and sync.

Step 19 — Connect the drawer to navigation

Open MainActivity.kt. Add this import at the top:

```
import androidx.navigation.ui.setupWithNavController
```

In onCreate(), find:

```
    appBarConfiguration = AppBarConfiguration(navController.graph)
```

Replace it with:

```
    appBarConfiguration = AppBarConfiguration(
        setOf(R.id.dashboardFragment, R.id.addTenantFragment, R.id.viewTenantsFragment),
        binding.drawerLayout
    )
```

Then, underneath setupActionBarWithNavController(navController, appBarConfiguration), add:

```
    binding.navView.setupWithNavController(navController)
```

What changed:

Code	What it does
setOf(...)	Lists the **top-level destinations**: the screens you reach from the drawer
binding.drawerLayout	Tells the App Bar there's a drawer, so it shows ☰ instead of ← on top-level screens
navView.setupWithNavController	Makes tapping a drawer item navigate to the destination with the same ID, and highlights the current screen

> **Question: What is a top-level destination?**
>
> A main screen of the app. On a top-level screen, the App Bar shows the ☰ drawer icon. On any other screen, it shows the ← back arrow. Until now, only the Dashboard was top-level. Now all three drawer screens are.
>
> **Question: What would happen if I listed only the Dashboard in setOf(...)?**
>
> Add Tenant and View Tenants would show a ← back arrow instead of ☰. The drawer would still open by swiping from the left edge, but the button wouldn't be there.

> **Question: Do I need to change onSupportNavigateUp()?**
> No. navController.navigateUp(appBarConfiguration) already knows about the drawer from appBarConfiguration, so tapping ☰ opens the drawer and tapping ← goes back.

Step 20 — Show the user's email in the drawer header

Still in MainActivity.kt, add this import:

```
import com.example.tenantmanagementsystemgroupa.databinding.NavHeaderBinding
```

At the end of onCreate(), after binding.navView.*setupWithNavController*(navController) add:

```
    val email = intent.getStringExtra("EMAIL")

    val headerBinding = NavHeaderBinding.bind(binding.navView.getHeaderView(0))

    headerBinding.headerEmailTextView.text = email ?: "Signed in"
```

> **Question: Why bind(...) and not inflate(...) like before?**
> inflate creates a new view from XML. The header already exists: NavigationView created it from app:headerLayout. bind connects a binding to a view that already exists.
>
> **Question: What is getHeaderView(0)?**
> The first (and only) header in the drawer. A NavigationView can have more than one header, so we ask for the one at position 0.
>
> **Question: What does email ?: "Signed in" mean?**
> If email is null (for example, if MainActivity was opened without an email), show "Signed in" instead. ?: is called the Elvis operator.

Step 21 — Your complete MainActivity.kt

```
package com.example.tenantmanagementsystemgroupa

import android.content.Intent
import android.os.Bundle
import android.util.Log
import android.view.Menu
import android.view.MenuItem
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.NavController
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.AppBarConfiguration
import androidx.navigation.ui.navigateUp
import androidx.navigation.ui.onNavDestinationSelected
import androidx.navigation.ui.setupActionBarWithNavController
import androidx.navigation.ui.setupWithNavController
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding
import com.example.tenantmanagementsystemgroupa.databinding.NavHeaderBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var navController: NavController
    private lateinit var appBarConfiguration: AppBarConfiguration

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("TENANT_LIFECYCLE", "MainActivity: onCreate")

        binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        // 1. Use our Toolbar as the App Bar
        setSupportActionBar(binding.toolbar)

        // 2. Get the NavController from the NavHost
        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
```

```
        navController = navHostFragment.navController

        // 3. Top-level destinations + the drawer
        appBarConfiguration = AppBarConfiguration(
            setOf(R.id.dashboardFragment, R.id.addTenantFragment, R.id.viewTenantsFragment),
            binding.drawerLayout
        )

        // 4. Connect the App Bar and the drawer to navigation
        setupActionBarWithNavController(navController, appBarConfiguration)

        binding.navView.setupWithNavController(navController)

        // 5. Show the user's email in the drawer header.
        //    Read the "EMAIL" extra that LoginActivity attached (Step 15)
        val email = intent.getStringExtra("EMAIL")

        // The header view already exists (app:headerLayout), so BIND to it
        val headerBinding = NavHeaderBinding.bind(binding.navView.getHeaderView(0))

        // headerEmailTextView ←  android:id="@+id/headerEmailTextView" in nav_header.xml
        headerBinding.headerEmailTextView.text = email ?: "Signed in"
    }

    // ☰ opens the drawer; ← goes back
    override fun onSupportNavigateUp(): Boolean {
        return navController.navigateUp(appBarConfiguration) || super.onSupportNavigateUp()
    }

    // Turn main_menu.xml into menu items in the App Bar
    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.main_menu, menu)
        return true
    }

    // What happens when a menu item is tapped
    override fun onOptionsItemSelected(item: MenuItem): Boolean {

        if (item.itemId == R.id.action_logout) {

            val intent = Intent(this, LoginActivity::class.java)

            intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK

            startActivity(intent)

            return true
        }

        return item.onNavDestinationSelected(navController) || super.onOptionsItemSelected(item)
    }

    // ── LIFECYCLE LOGS ──────────────────────────────────────────────────
    // Keep your onStart(), onResume(), onPause(), onStop(),
    // onRestart() and onDestroy() methods from Practical 2, Part A here.
    // (They are not repeated in this guide to save space.)
}
```

Question: binding.drawerLayout or binding.navView is red.
The IDs in activity_main.xml must be exactly drawerLayout and navView. After fixing them, run **Build → Rebuild Project**.

Question: NavHeaderBinding is red.
The header file must be named nav_header.xml, which generates NavHeaderBinding. Rebuild the project if the name is correct.

Run the complete app

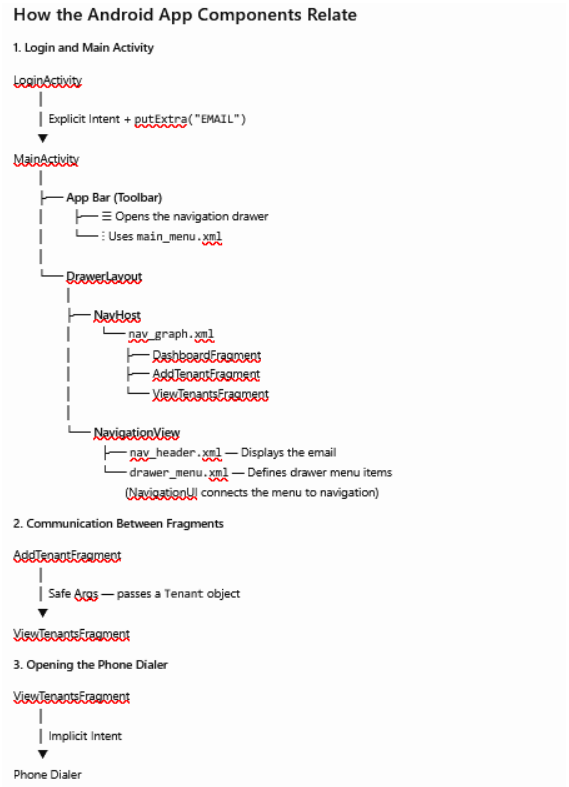
1. Log in with an email. The Dashboard shows ☰ on the left of the App Bar and ⋮ on the right. Logcat shows MainActivity and DashboardFragment starting.

1. Tap ☰. The drawer slides in with **your email** in the header and **Dashboard** highlighted.
2. Choose **Add Tenant**. Add John Kamau and tap **SAVE**. View Tenants shows him under "Just added".
3. **Rotate** the phone. He is still shown under "Just added".
4. Tap **CALL TENANT**. The dialer opens with his number. Return to the app.
5. Press **Back**. You're on the Dashboard, which now shows 1 tenant and his rent.
6. Open ⋮ and choose **View Tenants**. All tenants are listed, with no "Just added" section.
7. Open ⋮ and choose **Logout**. You're back on Login. Press Back: the app closes.

Final project structure

```
TenantManagementSystem
└── app
    ├── kotlin+java
    │   └── com.example.tenantmanagementsystemgroupa
    │       ├── LoginActivity.kt
    │       ├── RegisterActivity.kt
    │       ├── MainActivity.kt
    │       ├── DashboardFragment.kt
    │       ├── AddTenantFragment.kt
    │       ├── ViewTenantsFragment.kt
    │       ├── Tenant.kt
    │       └── TenantRepository.kt
    └── res
        ├── layout
        │   ├── activity_login.xml
        │   ├── activity_register.xml
        │   ├── activity_main.xml
        │   ├── nav_header.xml
        │   ├── fragment_dashboard.xml
        │   ├── fragment_add_tenant.xml
        │   └── fragment_view_tenants.xml
        ├── menu
        │   ├── main_menu.xml
        │   └── drawer_menu.xml
        └── navigation
            └── nav_graph.xml
```

How the pieces relate:



Common errors

What you see	Cause	Fix
...Directions or ...Args is red	Project not built after editing the nav graph	**Build → Make Project**
Build error about argType or Tenant when building the nav graph	Tenant is not Serializable	Check Tenant.kt has import java.io.Serializable and : Serializable (Practical 2, Step 9)
already has an action bar supplied by the window decor	Theme has its own action bar	Use a .NoActionBar theme
App Bar hidden under the status bar	Missing fitsSystemWindows	Add android:fitsSystemWindows="true" to the root
Back arrow in the App Bar does nothing	onSupportNavigateUp missing	Add it (Step 2)
navigation destination ... is unknown to this NavController	Wrong action used, or tapped twice quickly	Check the action starts from the current screen
Back from View Tenants returns to the form	No popUpTo on the action	Add app:popUpTo (Step 9)
NullPointerException on binding in a Fragment	Used binding after onDestroyView or before onCreateView	Only use binding inside onViewCreated and click listeners
Blank screen under the App Bar	Wrong app:navGraph or android:name on the FragmentContainerView	Compare with Step 1
No ⋮ icon in the App Bar	onCreateOptionsMenu missing, or returns false	Add it (Step 13)

What you see	Cause	Fix
Menu item does nothing	Menu ID doesn't match a destination ID	Make the IDs identical to those in nav_graph.xml
Back after Logout returns to the Dashboard	Missing Intent flags	Add FLAG_ACTIVITY_NEW_TASK or FLAG_ACTIVITY_CLEAR_TASK
Drawer covers the screen permanently	layout_gravity="start" missing on NavigationView	Add it (Step 18)
Drawer appears behind the screen	NavigationView is not the last child	Move it below the LinearLayout
App Bar shows ← instead of ☰	Destination not in setOf(...), or drawer not passed	Check Step 19
Tapping ☰ does nothing	onSupportNavigateUp missing	Keep it (Step 21)
Tapping a drawer item does nothing	setupWithNavController missing, or ID mismatch	Check Step 19 and the IDs in drawer_menu.xml
Header shows "Signed in" instead of the email	Email not sent from LoginActivity	Add putExtra("EMAIL", email) (Step 15)
NavigationView or MaterialToolbar is red	Material library missing	Add com.google.android.material:material and sync
App Bar hidden under the status bar	Missing fitsSystemWindows	Add it to the DrawerLayout and the LinearLayout

---

Try it yourself

1. **More Fragment logs.** Add lifecycle logs to AddTenantFragment. Navigate Dashboard → Add Tenant → Save and explain every line you see.
2. **Check the phone number.** In Add Tenant, refuse to save if the phone number has fewer than 10 digits, with binding.phoneEditText.error = "Enter a valid phone number".
3. **A fourth destination.** Create an AboutFragment, add it to the nav graph and to drawer_menu.xml with a matching ID. It should open from the drawer with no new Kotlin code in MainActivity.
4. **Logout in the drawer.** Add a second <group> to drawer_menu.xml with a Logout item, and handle it with binding.navView.setNavigationItemSelectedListener. For other items, call item.onNavDestinationSelected(navController) and then binding.drawerLayout.closeDrawers().
5. **Bottom navigation.** Instead of a drawer, try a BottomNavigationView with app:menu="@menu/drawer_menu", connected with binding.bottomNav.setupWithNavController(navController).

What you learned today

```
STEP 1   NavHost, App Bar, NavController and Safe Args
       "I can move between screens and pass whole objects safely."
            ↓
STEP 2   Fragment lifecycle
       "I know a Fragment's view can be destroyed while the Fragment lives on."
            ↓
STEP 3   Custom navigation behaviour
       "I can control the back stack and add animations."
            ↓
STEP 4   App Bar, options menu and navigation drawer
       "I can connect menus and a drawer to my nav graph with NavigationUI."
```

App Navigation: the whole picture

```
Practical 1  Intents                    Moving between Activities, sending data,
                                        and asking other apps for help
    ↓
Practical 2  Lifecycles + Fragments     The Activity lifecycle, saving state,
                                        Fragments, a nav graph and a NavHost
    ↓
Practical 3  Navigation + Drawer        Safe Args, the Fragment lifecycle,
                                        the back stack, menus and a drawer
```

Coming up next

We will explore app architecture and collaborative development, working in groups to complete the Tenant Management System. You will learn to use Git for version control, collaborate by pushing and pulling changes, and generate an APK that can be shared and installed on other devices.